

Filtraggio dei pacchetti e NAT

Indice dei contenuti

- 1. Testo
- 2. Suggerimenti
 - 2.1. Manuali di riferimento di sistema
 - 2.2. Ambiente virtuale
 - 2.3. Default deny
 - 2.4. Filtraggio dei pacchetti
 - 2.5. NAT
 - 2.6. File da consegnare
- 3. Soluzione
- Licenze

1. Testo

Configura un firewall Linux usando `iptables`. Il firewall ha due interfacce:

NIC	Indirizzo di rete	IP firewall	Ambito
eth0	203.0.113.0/24	203.0.113.1	Pubblico
eth1	10.20.30.0/24	10.20.30.1	Privato

Gli host sulla rete `10.20.30.0/24` usano questo firewall come loro gateway predefinito. L'host `10.20.30.100` esegue un web server che supporta l'Hypertext Transfer Protocol (HTTP) sulla porta `8080` e HTTP Secure (HTTPS) sulla porta `8443`.

Applica le seguenti regole:

Tabella	Chain	Regola
<code>filter,nat</code>	<code>*</code>	Svuota tutte le regole esistenti
<code>filter</code>	<code>INPUT, FORWARD</code>	Scarta tutto ciò che non è esplicitamente consentito
<code>filter</code>	<code>INPUT</code>	Consenti i pacchetti ICMP ricevuti su <code>eth1</code>
<code>filter</code>	<code>INPUT</code>	Consenti i pacchetti SSH (<code>tcp/22</code>) ricevuti su <code>eth1</code>
<code>filter</code>	<code>FORWARD</code>	Consenti i pacchetti HTTP (<code>tcp/80</code>) e HTTPS (<code>tcp/443</code>) ricevuti su <code>eth0</code> e <code>eth1</code>
<code>filter</code>	<code>FORWARD</code>	Consenti i pacchetti con stato <code>ESTABLISHED,RELATED</code>
<code>nat</code>	<code>POSTROUTING</code>	MASQUERADE i pacchetti in uscita da <code>eth0</code> affinché gli host privati ricevano risposte da Internet
<code>nat</code>	<code>PREROUTING</code>	DNAT per i pacchetti HTTP (<code>tcp/80</code>) ricevuti su <code>eth0</code> , inoltrandoli a <code>10.20.30.100:8080</code> e assicurandosi che possano raggiungere quell'host
<code>nat</code>	<code>PREROUTING</code>	DNAT per i pacchetti HTTPS (<code>tcp/443</code>) ricevuti su <code>eth0</code> , inoltrandoli a <code>10.20.30.100:8443</code> e assicurandosi che possano raggiungere quell'host

Usa questo template:

```
# nome e cognome:
# matricola:
```

2. Suggerimenti

2.1. Manuali di riferimento di sistema

`man` visualizza i manuali di riferimento del sistema. Ad esempio, `man:iptables(8)` si traduce in

```
$ man 8 iptables
```

Vedi anche `man:iptables-extensions(8)` per la documentazione dei moduli di corrispondenza come `conntrack` (`--ctstate`).

2.2. Ambiente virtuale

Gli script in `code/up.sh` e `code/down.sh` configurano e smontano la topologia descritta nel §1 usando `podman`. Rendili eseguibili

```
$ chmod +x up.sh down.sh
```

Per avviare l'ambiente

```
$ ./up.sh
```

Per smontarlo

```
$ ./down.sh
```

⚠ Warning

Le regole `iptables` di un container vivono nel suo namespace di rete e vengono scartate quando il container viene rimosso. Rieseguire `./up.sh` dopo `./down.sh` fornisce un firewall pulito.

Apri tre shell in terminali separati, uno per container

```
$ podman exec -it firewall bash
```

```
$ podman exec -it client bash
```

```
$ podman exec -it web bash
```

Prima di scrivere qualsiasi regola, verifica che la topologia sia collegata correttamente. Ad esempio, l'output seguente mostra quale indirizzo Internet Protocol (IP) è assegnato a quale interfaccia sul `firewall`:

```
ubuntu@firewall:~$ ip a
1: lo: [...]
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth1@if4: [...]
   link/ether 96:95:c5:05:68:ee brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet 10.20.30.1/24 brd 10.20.30.255 scope global eth1
       valid_lft forever preferred_lft forever
   inet6 fe80::9495:c5ff:fe05:68ee/64 scope link
       valid_lft forever preferred_lft forever
3: eth0@if6: [...]
   link/ether 3e:56:eb:e1:7c:98 brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet 203.0.113.1/24 brd 203.0.113.255 scope global eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::3c56:ebff:fee1:7c98/64 scope link
       valid_lft forever preferred_lft forever
```

Un host invia tutto il traffico che non corrisponde a un percorso più specifico al suo gateway predefinito. Per controllare il gateway predefinito su un host

```
ubuntu@client:~$ ip r
default via 203.0.113.1 dev eth0
203.0.113.0/24 dev eth0 proto kernel scope link src 203.0.113.50
```

```
ubuntu@web:~$ ip r
default via 10.20.30.1 dev eth0
10.20.30.0/24 dev eth0 proto kernel scope link src 10.20.30.100
```

firewall è il gateway predefinito sia per web che per client. Poiché

```
ubuntu@firewall:~$ cat /proc/sys/net/ipv4/ip_forward
1
```

firewall ha l'inoltro IP abilitato e inoltrerà i pacchetti. Quindi, da web, esegui il ping di client

```
ubuntu@web:~$ ping 203.0.113.50
PING 203.0.113.50 (203.0.113.50) 56(84) bytes of data.
64 bytes from 203.0.113.50: icmp_seq=1 ttl=63 time=1.15 ms
64 bytes from 203.0.113.50: icmp_seq=2 ttl=63 time=0.235 ms
^C
--- 203.0.113.50 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1006ms
rtt min/avg/max/mdev = 0.235/0.691/1.148/0.456 ms
```

Per osservare l'inoltro, esegui tcpdump su firewall

```
ubuntu@firewall:~$ tcpdump -i any icmp
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
14:00:55.699374 eth0 In IP 203.0.113.50 > 10.20.30.100: ICMP echo request, id 9, seq 10, length 64
14:00:55.699406 eth1 Out IP 203.0.113.50 > 10.20.30.100: ICMP echo request, id 9, seq 10, length 64
14:00:55.699470 eth1 In IP 10.20.30.100 > 203.0.113.50: ICMP echo reply, id 9, seq 10, length 64
14:00:55.699478 eth0 Out IP 10.20.30.100 > 203.0.113.50: ICMP echo reply, id 9, seq 10, length 64

[...]
```

- `-i any` indica a `tcpdump` di ascoltare su tutte le interfacce contemporaneamente; puoi sostituire `any` con un nome di interfaccia specifico (es. `-i eth0`) per catturare il traffico solo su quell'interfaccia

Note

`tcpdump` bufferizza l'output prima di stamparlo. Se non vedi subito i pacchetti, attendi un momento.

2.3. Default deny

Il default deny si articola in due fasi: svuotare tutte le chain in `filter` e `nat` per eliminare le regole preesistenti, poi impostare la policy predefinita di `INPUT` e `FORWARD` su `DROP`. Lascia `OUTPUT` al valore predefinito `ACCEPT`: `firewall` è considerato attendibile per inviare risposte e qualsiasi traffico generato localmente. La tabella `nat` non filtra, quindi le policy delle sue chain rimangono al valore predefinito `ACCEPT`. Pertanto

```
ubuntu@firewall:~$ iptables -F
ubuntu@firewall:~$ iptables -t nat -F
ubuntu@firewall:~$ iptables -P INPUT DROP
ubuntu@firewall:~$ iptables -P FORWARD DROP
```

Per elencare tutte le regole

```
ubuntu@firewall:~$ iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination

Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
```

Note

Come ogni altro comando `iptables`, si applica alla tabella specificata. Se non viene specificata alcuna tabella, il valore predefinito è `filter`. Ad esempio, usa `iptables -L -t nat` per elencare tutte le regole della tabella `nat`.

Warning

Con queste policy attive, qualsiasi pacchetto non esplicitamente consentito viene scartato. In particolare, `web` e `client` non potranno più fare il ping di `firewall` (`INPUT DROP`), né potranno raggiungersi a vicenda (`FORWARD DROP`).

2.4. Filtraggio dei pacchetti

Warning

Questa sezione utilizza porte ipotetiche per illustrare la meccanica di `FORWARD`. NON è ciò che è richiesto dal §1.

Supponi di voler permettere a `client` di raggiungere `web` sulla porta `9999`. Un primo tentativo è aggiungere una regola a `FORWARD` che consenta `tcp/9999` in arrivo su `eth0` e in uscita su `eth1`:

```
ubuntu@firewall:~$ iptables \
-A FORWARD \
-i eth0 \
-o eth1 \
-p tcp \
--dport 9999 \
-j ACCEPT
```

Questa regola da sola non è sufficiente. Copre solo i pacchetti Transmission Control Protocol (TCP) destinati alla porta `9999` da `eth0` a `eth1`, ma non il traffico di ritorno. `client` può raggiungere `web`, ma le risposte di `web` vengono scartate da `firewall`.

La soluzione è una regola con stato (stateful) che consente i pacchetti appartenenti a una connessione già vista dal firewall:

```
ubuntu@firewall:~$ iptables \
-A FORWARD \
-m conntrack \
--ctstate ESTABLISHED,RELATED \
-j ACCEPT
```

Per verificare, metti in ascolto `web`

```
ubuntu@web:~$ nc -l -p 9999
```

e connettiti da `client`

```
ubuntu@client:~$ nc 10.20.30.100 9999
```

e i byte digitati nella shell di `client` appariranno sul listener di `web`.

In un deployment reale, `client` non conoscerebbe l'IP privato di `web`. Il §2.5 colma questa lacuna con `DNAT`.

2.5. NAT

⚠ Warning

Questa sezione aggiunge `DNAT` alle regole `FORWARD` del §2.4. Le porte rimangono ipotetiche — NON sono quelle richieste dal §1.

`DNAT` riscrive la destinazione di un pacchetto in ingresso. Viene eseguito in `PREROUTING`, prima della decisione di routing e prima di `FORWARD`, in modo che il pacchetto entri nella tabella `filter` con la sua destinazione riscritta. Ad esempio, per inoltrare `tcp/9000` in arrivo su `eth0` a `10.20.30.100:9999`

```
ubuntu@firewall:~$ iptables \
-t nat \
-A PREROUTING \
-i eth0 \
-p tcp \
--dport 9000 \
-j DNAT \
--to-destination 10.20.30.100:9999
```

⚠ Warning

Ogni regola `DNAT` deve essere abbinata a un permesso `FORWARD` sulla porta post-DNAT — senza di esso, il pacchetto riscritto incappa nel `DROP` predefinito.

`SNAT` riscrive l'indirizzo sorgente dei pacchetti in uscita. Viene eseguito in `POSTROUTING`, dopo la decisione di routing e dopo `FORWARD`, in modo che le risposte vengano reindirizzate a `firewall` piuttosto che a un indirizzo privato che l'host remoto non può raggiungere. Ad esempio, per riscrivere la sorgente di ogni pacchetto che lascia `eth0` con l'IP pubblico del `firewall`

```
ubuntu@firewall:~$ iptables \
-t nat \
-A POSTROUTING \
-o eth0 \
-j SNAT \
--to-source 203.0.113.1
```

`MASQUERADE` è una variante di `SNAT` che viene eseguita anch'essa in `POSTROUTING` e usa automaticamente l'indirizzo attualmente assegnato all'interfaccia in uscita. È la scelta giusta quando quell'indirizzo non è noto staticamente. Ad esempio, per mascherare tutto il traffico in uscita da `eth0`

```
ubuntu@firewall:~$ iptables \
-t nat \
-A POSTROUTING \
-o eth0 \
-j MASQUERADE
```

2.6. File da consegnare

Il file da consegnare è un file di testo semplice contenente tutti i comandi `iptables`, che inizia con il template del §1.

💡 Tip

Per ispezionare le regole attualmente caricate nel kernel e confrontarle con il tuo file:

```
$ iptables -S
$ iptables -t nat -S
```

3. Soluzione

Questo esercizio è stato proposto il [9 febbraio 2026](#).

Licenze

Contenuto	Licenza
Codice	MIT License
Testo	Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International